

Documentación del Código

Unidad 5 – Programación Orientada a Objetos con Clases
Archivo: **U5_Ejercicios_en_clase.cpp**

Vista General del Archivo

El archivo **U5_Ejercicios_en_clase.cpp** contiene los 5 ejercicios de la Unidad 5 en un único archivo sin programación modular (sin archivos .h separados). Cada ejercicio define una o más clases seguidas de una sección del **main()** que las instancia y utiliza.

Las directivas incluidas al inicio son:

```
#include <iostream> // entrada/salida estándar (cout, cin)
#include <string> // tipo string y sus métodos
#include <cstdlib> // rand(), srand()
#include <ctime> // time(0) para la semilla aleatoria
#include <cmath> // funciones matemáticas (pow, sqrt, etc.)
using namespace std; // evita escribir std:: antes de cada símbolo
```

Ejercicio 1 – Clase Tanque

Propósito

Modela un tanque de almacenamiento con capacidad máxima fija. Permite agregar y extraer contenido con restricciones para no superar la capacidad ni quedar en valores negativos.

Atributos

Atributo	Tipo	Acceso	Descripción
contenido	double	private	Cantidad actual almacenada en el tanque.
capacidad	static const int	public	Capacidad máxima (5000). Única para todas las instancias.

Métodos

Tanque() – Constructor

Inicializa **contenido** en 0.0. Al crear un objeto Tanque sin argumentos, el tanque comienza completamente vacío.

```
Tanque t; // contenido = 0.0
```

double getContenido()

Devuelve el valor actual de **contenido**. Es el único medio de leer el estado interno desde fuera de la clase.

void agregar(double cantidad)

Suma **cantidad** al contenido actual. Si la suma supera **capacidad**, el contenido queda exactamente en el valor de capacidad (no se desborda).

```
contenido = (contenido + cantidad > capacidad) ? (double)capacidad : contenido + cantidad;
```

void sacar(double cantidad)

Resta **cantidad** del contenido. Si **cantidad** es mayor que lo que hay, el contenido queda en 0.0 (no se producen valores negativos).

```
contenido = (cantidad > contenido) ? 0.0 : contenido - cantidad;
```

void sacaMitad()

Divide el contenido entre 2. Si el tanque está vacío (contenido == 0), no hace nada, evitando operaciones innecesarias.

```
if (contenido > 0.0) contenido /= 2.0;
```

Uso en main()

Se crea un tanque, se agregan 100 unidades y se ejecuta un bucle **while** que llama a **sacaMitad()** repetidamente mientras el contenido sea ≥ 1.0 . La condición asegura que el bucle termina porque cada iteración reduce el contenido a la mitad (convergencia geométrica hacia cero).

```
Tanque t;
t.agregar(100.0);
while (t.getContenido() >= 1.0) {
    t.sacaMitad();
}
cout << "El contenido final es: " << t.getContenido();
```

Ejercicio 2 – Clase Sala

Propósito

Representa una sala para eventos. Calcula automáticamente la superficie, la capacidad máxima de asistentes y el costo de alquiler según si la sala está equipada o no.

Atributos

Atributo	Tipo	Descripción
nombre	string	Nombre identificador de la sala.
largo	int	Longitud de la sala en metros.
ancho	int	Anchura de la sala en metros.
equipado	bool	true si la sala tiene equipamiento incluido.

Métodos

Sala(string n, int l, int a, bool e) – Constructor

Inicializa todos los atributos con los valores proporcionados. No existe constructor por defecto; siempre hay que dar nombre, dimensiones y equipamiento.

```
Sala s1("Sala 1", 12, 8, true); // 12m x 8m, equipada
```

Getters inline

Los métodos **getNombre()**, **getAncho()**, **getLargo()** y **getEquipado()** son funciones *inline*: se expanden en el lugar de la llamada en tiempo de compilación, evitando el overhead de una llamada normal.

int calcularArea()

Devuelve la superficie en m² multiplicando largo por ancho.

```
return largo * ancho; // ej: 12 * 8 = 96 m2
```

int calcularCapacidad()

Divide el área entre 1.5 (cada asistente necesita 1.5 m²) y trunca al entero inferior. La conversión explícita (**int**) realiza ese truncamiento.

```
return (int)(calcularArea() / 1.5); // ej: 96 / 1.5 = 64 personas
```

int calcularAlquiler()

Multiplica el área por \$45 si la sala está equipada, o por \$32 en caso contrario. Usa el operador ternario para seleccionar el precio en una sola línea.

```
return calcularArea() * (equipado ? 45 : 32);
```

void mostrarDatos()

Imprime en consola todos los campos más los tres valores calculados (área, capacidad, alquiler). Es el método de presentación del objeto.

Uso en main()

Se crean tres salas y se almacenan en un arreglo. Primero se muestran los datos completos de Sala 1. Luego, un doble bucle **for** itera sobre 4 eventos (con 200, 50, 100 y 150 asistentes) y, para cada evento, recorre las 3 salas imprimiendo solo las que tienen capacidad suficiente.

```
for (int i = 0; i < 4; i++)
for (int j = 0; j < 3; j++)
if (salas[j].calcularCapacidad() >= eventos[i])
cout << salas[j].getNombre() << "...";
```

Ejercicio 3 – Clase Persona

Propósito

Modela a una persona con sus datos biométricos. Calcula el Índice de Masa Corporal (IMC), verifica la mayoría de edad y genera automáticamente un identificador único (ID) compuesto por las iniciales del nombre más un número aleatorio de 3 cifras.

Atributos privados

Atributo	Tipo	Descripción
nombre	string	Nombre completo de la persona.
ID	string	Identificador generado automáticamente al construir el objeto.
edad	int	Edad en años.
sexo	char	'H' para hombre, 'M' para mujer.
peso	double	Peso en kilogramos.
altura	double	Altura en metros.

Métodos privados (solo visibles dentro de la clase)

void comprobarSexo(char s)

Valida que el carácter sea 'H' o 'M'. Si el usuario ingresa cualquier otro valor, se asigna 'H' por defecto. Esto garantiza que el atributo **sexo** siempre sea válido.

```
sexo = (s == 'H' || s == 'M') ? s : 'H';
```

void generalID()

Recorre el nombre letra a letra y extrae la primera letra de cada palabra (detectando espacios). Luego genera un número aleatorio entre 100 y 999 y lo concatena con las iniciales para formar el ID.

```
int num = rand() % 900 + 100; // 100 a 999
// recorre el nombre y toma la primera letra de cada palabra
ID = iniciales + to_string(num);
```

Ejemplo: nombre "Juan Carlos" → iniciales "JC" → ID podría ser "JC473".

Métodos públicos

Constructor Persona(string, int, char, double, double)

Inicializa todos los atributos. Internamente llama a **comprobarSexo()** para validar el sexo y a **generalID()** para crear el identificador. El ID no se puede cambiar desde fuera (no hay setter para él).

int calcularIMC()

Calcula el IMC con la fórmula **peso / (altura × altura)** y retorna:

- -1 si $IMC < 20$ → persona con bajo peso
- 0 si $20 \leq IMC \leq 25$ → persona en peso ideal
- 1 si $IMC > 25$ → persona con sobrepeso

```
double imc = peso / (altura * altura);
```

bool esMayorDeEdad()

Devuelve **true** si la edad es mayor o igual a 18 años, **false** en caso contrario.

void mostrar()

Imprime en consola todos los atributos del objeto, incluido el ID generado.

Getters y Setters

Hay getters y setters para **nombre**, **edad**, **sexo**, **peso** y **altura**. El setter de **sexo** pasa por **comprobarSexo()** para mantener la validación. **ID no tiene setter**: es de solo lectura.

Uso en main()

El programa solicita datos por teclado para 3 personas usando una mezcla de **cin** (para números y char) y **getline()** (para nombres con espacios). Se necesita **cin.ignore()** entre ambos para descartar el salto de línea pendiente en el buffer.

```
cin.ignore(); // limpia el '\n' antes de getline
getline(cin, pNombres[i]); // lee el nombre completo con espacios
```

Los 3 objetos se crean con **new** (memoria dinámica) y al final se liberan con **delete** para evitar fugas de memoria.

Ejercicio 4 – Clases Atleta y Carrera

Clase Atleta

Representa a un competidor con nombre, nacionalidad, número de dorsal y tiempo. Tiene dos constructores: uno por defecto (necesario para crear arrays) y otro con parámetros para inicializar con datos reales.

operator<< (sobrecarga del operador de salida)

Permite usar directamente **cout << atleta** sin necesidad de llamar a un método mostrar(). La palabra clave **friend** le da acceso a los atributos privados de la clase desde fuera de ella.

```
friend ostream& operator<<(ostream& os, Atleta& a) { ... }
cout << ganador; // imprime automáticamente todos los datos
```

Clase Carrera

Gestiona un conjunto de atletas para una carrera de una distancia dada. El array de competidores es dinámico (creado con **new**), lo que permite que el tamaño se decida en tiempo de ejecución.

Carrera(int distancia, int n) – Constructor

Reserva memoria para **n** atletas con **new Atleta[n]**. El índice **indx** empieza en 0 y avanza con cada atleta agregado.

```
competidores = new Atleta[n];
```

~Carrera() – Destructor

Libera el array dinámico con **delete[]** cuando el objeto sale de scope. Es fundamental para evitar fugas de memoria.

```
~Carrera() { delete[] competidores; }
```

void addAtleta(Atleta& a)

Copia el atleta en la posición **indx** del array y luego incrementa el índice. La verificación **if (indx < numAtletas)** evita escribir fuera de los límites.

Atleta ganador()

Recorre el array y compara tiempos. Quien tenga el **menor tiempo** gana (en atletismo se gana con el tiempo más bajo). Devuelve una copia del objeto ganador.

```
if (competidores[i].getTiempo() < g.getTiempo()) g = competidores[i];
```

Uso en main()

```
Carrera carr1(1000, 3); // 1000 m, máximo 3 atletas
Atleta a1("Pancho", 1, "Mexico", 9.1f);
Atleta a2("Chacho", 3, "Colombia", 3.6f);
Atleta a3("Robert", 2, "Chile", 6.9f);
carr1.addAtleta(a1); carr1.addAtleta(a2); carr1.addAtleta(a3);
Atleta gan = carr1.ganador();
cout << gan; // imprime datos de Chacho (tiempo 3.6s)
```

Ejercicio 5 – Clases Vagon y Tren

Clase Vagon

Representa un vagón de tren con 40 asientos que pueden estar ocupados (true) o vacantes (false). El vagón tiene una clase: **false = primera clase**, **true = segunda clase**.

Constructores

Vagon(): primera clase por defecto, todos los asientos en false (vacantes). **Vagon(bool c)**: permite especificar la clase al crear el vagón.

```
for (int i = 0; i < 40; i++) asientos[i] = false; // todos vacantes
```

void ocupar()

Llena los asientos con probabilidad aleatoria. Usa **rand() % 100** para generar un número entre 0 y 99 y compara con el umbral:

- Primera clase (clase=false): 10% de probabilidad → umbrales menores a 10
- Segunda clase (clase=true): 40% de probabilidad → umbrales menores a 40

```
int prob = clase ? 40 : 10;
if ((rand() % 100) < prob) asientos[i] = true;
```

int asientosOcupados()

Cuenta cuántos asientos tienen el valor **true** e itera todo el array. Se usa tanto en el operador de salida como en el cálculo de ventas.

operator<< (sobrecarga de salida)

Imprime la clase del vagón, el número de asientos ocupados y el estado individual de cada uno de los 40 asientos.

Clase Tren

Agrupar múltiples vagones de primera y segunda clase, con información de ruta y precios de tickets. Usa un array dinámico de objetos Vagon.

Tren(int nP, int nS, string sal, string dest, int pP, int pS) – Constructor

Crea el array de vagones y asigna la clase a cada uno: los primeros **nP** son de primera, los siguientes **nS** son de segunda.

```
vagones = new Vagon[nP + nS];
for (int i = 0; i < nP; i++) vagones[i] = Vagon(false); // primera
for (int i = nP; i < nP + nS; i++) vagones[i] = Vagon(true); // segunda
```

~Tren() – Destructor

Libera el array dinámico de vagones al destruirse el objeto.

void llenar()

Llama a **ocupar()** en cada vagón, llenando aleatoriamente todos los asientos del tren en una sola operación.

int totalVentas()

Suma el producto de asientos ocupados por precio, separando vagones de primera y segunda clase para aplicar la tarifa correcta a cada uno.

```
total += vagones[i].asientosOcupados() * precioPrimera; // primera
total += vagones[i].asientosOcupados() * precioSegunda; // segunda
```

Uso en main()

```
Vagon v;
v.ocupar();
v.setClase(true);
cout << v; // muestra estado del vagón
Tren tren(4, 7, "Aguascalientes", "Calvillo", 20, 10);
tren.llenar();
cout << tren.totalVentas(); // total de ventas en $
```

Semilla aleatoria: srand(time(0))

La función **rand()** produce la misma secuencia de números en cada ejecución si no se cambia la semilla. **srand(time(0))** usa el reloj del sistema como semilla, garantizando secuencias distintas en cada ejecución. Se llama **una sola vez** al inicio de main() para no reiniciar la secuencia.

```
srand(time(0)); // al inicio de main(), una única vez
```

Memoria dinámica: new / delete

Cuando el tamaño de un array se conoce en tiempo de ejecución (como en Carrera y Tren), se usa **new** para reservar memoria en el heap. Es obligatorio liberar esa memoria con **delete[]** cuando ya no se necesita, habitualmente en el destructor.

```
competidores = new Atleta[n]; // reserva en heap
delete[] competidores; // libera en el destructor
```

cin.ignore() y getline()

cin >> deja un salto de línea ('\n') en el buffer de entrada. Si después se llama a **getline()**, este lee ese salto de línea vacío en lugar del texto del usuario. **cin.ignore()** descarta ese carácter pendiente, asegurando que **getline()** espere la entrada real.

```
cin >> edad;
cin.ignore(); // descarta el '\n' que queda
getline(cin, nombre); // ahora lee correctamente
```

Operador ternario

Es una forma compacta de escribir una asignación condicional. La sintaxis es **condición ? valor_si_true : valor_si_false**.

```
contenido = (contenido + cantidad > capacidad) ? capacidad : contenido + cantidad;
```

Equivale a:

```
if (contenido + cantidad > capacidad)
    contenido = capacidad;
else
    contenido = contenido + cantidad;
```

inline

Los métodos marcados como **inline** (o definidos dentro del cuerpo de la clase) le indican al compilador que expanda el código en el punto de llamada en lugar de generar una llamada a función. Es útil para métodos muy cortos como getters, ya que elimina el overhead de la llamada.

friend ostream& operator<<

Sobrecarga el operador << para que un objeto de la clase pueda imprimirse directamente con cout. La palabra clave **friend** le da acceso a los atributos privados. El parámetro **ostream&** se devuelve para permitir encadenamiento: **cout << a << b**.

```
cout << ganador; // llama al operator<<
cout << a1 << a2 << a3; // encadenamiento posible
```